

Building and Manipulating RDF Graphs

Simon Frost

Overview

This vignette covers how to build RDF graphs, add and remove triples, query for patterns, and use helper utilities like the `Describer` and `Resource` abstractions.

```
using RDFLib
ex = Namespace("http://example.org/")
```

```
Namespace("http://example.org/")
```

Creating Graphs

```
# Empty graph
g = RDFGraph()
println("Empty graph: ${length(g)} triples")

# Add triples one at a time
add!(g, Triple(ex("sun"), RDF.type, ex("Star")))
add!(g, Triple(ex("earth"), RDF.type, ex("Planet")))
add!(g, Triple(ex("earth"), ex("orbits"), ex("sun")))
add!(g, Triple(ex("mars"), RDF.type, ex("Planet")))
add!(g, Triple(ex("mars"), ex("orbits"), ex("sun")))
add!(g, Triple(ex("moon"), RDF.type, ex("Moon")))
add!(g, Triple(ex("moon"), ex("orbits"), ex("earth")))

println("After adding: ${length(g)} triples")
```

```
Empty graph: 0 triples
After adding: 7 triples
```

Querying Patterns

Use `triples()` with a pattern tuple (subject, predicate, object) where `nothing` is a wildcard.

```
# Find all planets
println("Planets:")
for t in triples(g, (nothing, RDF.type, ex("Planet")))
    println("  ", t.subject)
end

# Find what orbits the sun
println("\nOrbiting the sun:")
for t in triples(g, (nothing, ex("orbits"), ex("sun")))
    println("  ", t.subject)
end

# Find all facts about Earth
println("\nAbout Earth:")
for t in triples(g, (ex("earth"), nothing, nothing))
    println("  ", t.predicate, " → ", t.object)
end
```

Planets:

```
URIRef("http://example.org/earth")
URIRef("http://example.org/mars")
```

Orbiting the sun:

```
URIRef("http://example.org/earth")
URIRef("http://example.org/mars")
```

About Earth:

```
URIRef("http://www.w3.org/1999/02/22-rdf-syntax-ns#type") → URIRef("http://example.org/Planet")
URIRef("http://example.org/orbits") → URIRef("http://example.org/sun")
```

Convenience Query Functions

```
# subjects(graph, predicate, object) - find matching subjects
planets = collect(subjects(g, RDF.type, ex("Planet")))
println("Planets: ", planets)
```

```

# objects(graph, subject, predicate) - find matching objects
earth_props = collect(objects(g, ex("earth"), ex("orbits")))
println("Earth orbits: ", earth_props)

# predicates(graph, subject, object) - find matching predicates
rels = collect(predicates(g, ex("earth"), ex("sun")))
println("Earth→Sun relations: ", rels)

# predicate_objects - pairs of (predicate, object) for a subject
println("\nEarth properties:")
for (p, o) in predicate_objects(g, ex("earth"))
    println("  ", p, " → ", o)
end

```

```

Planets: URIRef[URIRef("http://example.org/earth"), URIRef("http://example.org/mars")]
Earth orbits: URIRef[URIRef("http://example.org/sun")]
Earth→Sun relations: URIRef[URIRef("http://example.org/orbits")]

```

Earth properties:

```

URIRef("http://www.w3.org/1999/02/22-rdf-syntax-ns#type") → URIRef("http://example.org/Planets")
URIRef("http://example.org/orbits") → URIRef("http://example.org/sun")

```

Removing Triples

```

println("Before removal: $(length(g)) triples")

# Remove a specific triple
remove!(g, (ex("moon"), ex("orbits"), ex("earth")))
println("After removing moon-orbits-earth: $(length(g)) triples")

# Remove all triples matching a pattern
remove!(g, (ex("moon"), nothing, nothing))
println("After removing all moon triples: $(length(g)) triples")

```

Before removal: 7 triples

After removing moon-orbits-earth: 6 triples

After removing all moon triples: 5 triples

Namespace Binding

Bind prefixes for cleaner serialization:

```
bind!(g, "ex", Namespace("http://example.org/"))
println(serialize(g, TurtleFormat()))
```

```
@prefix ex: <http://example.org/> .
@prefix owl: <http://www.w3.org/2002/07/owl#> .
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
@prefix skos: <http://www.w3.org/2004/02/skos/core#> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .

ex:earth a ex:Planet ;
    ex:orbits ex:sun .

ex:mars a ex:Planet ;
    ex:orbits ex:sun .

ex:sun a ex:Star .
```

Using the Describer

The `Describer` provides a fluent API for building graph descriptions:

```
g2 = RDFGraph()
bind!(g2, "schema", Namespace("http://schema.org/"))
schema = Namespace("http://schema.org/")

d = describe(g2, schema("julia-lang"))
rdf_type!(d, schema("SoftwareApplication"))
property!(d, schema("name"), Literal("Julia Programming Language"))
property!(d, schema("url"), URIRef("https://julia-lang.org"))
property!(d, schema("dateCreated"), Literal("2012-02-14"))
property!(d, schema("programmingLanguage"), Literal("Julia"))
properties!(d, schema("author"), [
    Literal("Jeff Bezanson"),
    Literal("Stefan Karpinski"),
    Literal("Viral B. Shah"),
    Literal("Alan Edelman")
])
```

```
])
```

```
println("Described $(length(g2)) triples:")  
println(serialize(g2, TurtleFormat()))
```

Described 9 triples:

```
@prefix ns1: <https://> .  
@prefix owl: <http://www.w3.org/2002/07/owl#> .  
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .  
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .  
@prefix schema: <http://schema.org/> .  
@prefix skos: <http://www.w3.org/2004/02/skos/core#> .  
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .
```

```
schema:juliaLang a schema:SoftwareApplication ;  
    schema:author "Jeff Bezanson",  
        "Stefan Karpinski",  
        "Viral B. Shah",  
        "Alan Edelman" ;  
    schema:dateCreated "2012-02-14" ;  
    schema:name "Julia Programming Language" ;  
    schema:programmingLanguage "Julia" ;  
    schema:url ns1:juliaLang.org .
```

Using Resources

The `Resource` abstraction provides object-like access to RDF data:

```
g3 = parse_rdf("""  
    @prefix ex: <http://example.org/> .  
    @prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .  
  
    ex:Dog a rdfs:Class ;  
        rdfs:label "Dog"@en ;  
        rdfs:comment "A domestic canine" ;  
        rdfs:subClassOf ex:Animal .  
  
    ex:fido a ex:Dog ;  
        rdfs:label "Fido" ;  
        ex:age 5 ;  
        ex:owner ex:alice .  
""")
```

```

""" , TurtleFormat())

fido = Resource(g3, ex("fido"))
println("Types: ", types(fido))
println("Label: ", label(fido))
println("Age: ", fido[ex("age")])
println("Owner: ", fido[ex("owner")])

```

```

Types: Identifier[URIRef("http://example.org/Dog")]
Label: Literal("Fido")
Age: Literal("5", datatype=URIRef("http://www.w3.org/2001/XMLSchema#integer"))
Owner: URIRef("http://example.org/alice")

```

Graph Operations

Merging

```

g_a = parse_rdf("""
    @prefix ex: <http://example.org/> .
    ex:alice ex:knows ex:bob .
    ex:alice ex:name "Alice" .
""", TurtleFormat())

g_b = parse_rdf("""
    @prefix ex: <http://example.org/> .
    ex:bob ex:knows ex:carol .
    ex:bob ex:name "Bob" .
""", TurtleFormat())

merged = merge_graphs(g_a, g_b)
println("Merged: $(length(merged)) triples")

```

```
Merged: 4 triples
```

Difference

```

in_both, in_a, in_b = graph_diff(g_a, g_b)
println("Only in A: ${length(in_a)}")
println("Only in B: ${length(in_b)}")
println("In both:   ${length(in_both)}")

```

```

Only in A: 2
Only in B: 2
In both:   0

```

Statistics

```

stats = graph_stats(merged)
for k in keys(stats)
    println(" $k: ${stats[k]}")
end

```

```

triples: 4
subjects: 2
predicates: 2
objects: 4
uri_refs: 5
blank_nodes: 0
literals: 2

```

Transitive Closure

Follow chains of properties transitively:

```

g4 = parse_rdf("""
    @prefix ex: <http://example.org/> .
    @prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .

    ex:Poodle rdfs:subClassOf ex:Dog .
    ex:Dog rdfs:subClassOf ex:Canine .
    ex:Canine rdfs:subClassOf ex:Mammal .
    ex:Mammal rdfs:subClassOf ex:Animal .
""", TurtleFormat())

```

```
# Find all superclasses of Poodle
supers = transitive_objects(g4, ex("Poodle"), RDFS("subClassOf"))
println("Poodle superclasses:")
for s in supers
    println("  ", s)
end
```

Poodle superclasses:

```
URIRef("http://example.org/Mammal")
URIRef("http://example.org/Poodle")
URIRef("http://example.org/Animal")
URIRef("http://example.org/Dog")
URIRef("http://example.org/Canine")
```

Concise Bounded Description

Extract a self-contained subgraph describing a specific node:

```
g5 = parse_rdf("""
    @prefix ex: <http://example.org/> .

    ex:alice ex:name "Alice" ;
        ex:address [ ex:city "London" ; ex:country "UK" ] ;
        ex:knows ex:bob .
    ex:bob ex:name "Bob" .
""", TurtleFormat())

cbd_graph = cbd(g5, ex("alice"))
println("CBD of alice ($(length(cbd_graph)) triples):")
println(serialize(cbd_graph, TurtleFormat()))
```

CBD of alice (5 triples):

```
@prefix ns1: <http://example.org/> .
@prefix owl: <http://www.w3.org/2002/07/owl#> .
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
@prefix skos: <http://www.w3.org/2004/02/skos/core#> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .

ns1:alice ns1:address [ ns1:city "London" ;
```



```

    ns1:country "UK" ] ;
ns1:knows ns1:bob ;
ns1:name "Alice" .

```

Skolemization

Replace blank nodes with deterministic URIs:

```

g6 = parse_rdf("""
    @prefix ex: <http://example.org/> .
    ex:alice ex:address [ ex:city "London" ] .
""", TurtleFormat())

println("Before skolemization:")
for t in g6; println(" ", t); end

skolemized = skolemize(g6)
println("\nAfter skolemization:")
for t in skolemized; println(" ", t); end

```

Before skolemization:

```

(BNode("b1"), URIRef("http://example.org/city"), Literal("London"))
(URIRef("http://example.org/alice"), URIRef("http://example.org/address"), BNode("b1"))

```

After skolemization:

```

(URIRef("https://rdflib.github.io/.well-known/genid/b1"), URIRef("http://example.org/city"))
(URIRef("http://example.org/alice"), URIRef("http://example.org/address"), URIRef("https://rdflib.github.io/.well-known/genid/b1"))

```

Isomorphism

Test whether two graphs are structurally identical (up to blank node renaming):

```

g7 = parse_rdf("""
    @prefix ex: <http://example.org/> .
    ex:a ex:p [ ex:q "hello" ] .
""", TurtleFormat())

g8 = parse_rdf("""
    @prefix ex: <http://example.org/> .

```

```
    ex:a ex:p [ ex:q "hello" ] .  
    "", TurtleFormat()  
  
println("Isomorphic: ", isomorphic(g7, g8))
```

Isomorphic: true